DAY 3 · MODULE 4 · 35 MIN

Middleware & Robust Agents

Add cross-cutting controls without burying them in tools or prompts

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/?tabs=python

Overview

- Middleware implements cross-cutting concerns
  - Logging
  - Security validation
  - Error handling
  - Transformation of results
- Separate from your core logic

When multiple middleware of the same type are registered, they form a chain where each calls the call_next() callback to continue processing.

call_next() does not take the context as an argument; middleware mutates the shared context object directly and then awaits call_next(). The call_next callback continues the middleware chain or executes the agent if it's the last middleware.

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/?tabs=python&pivots=programming-language-python

Three Types of Middleware

Type	Intercepts	Typical frequency
Agent middleware	Whole agent run	Once per run
Function (tool) middleware	One function/tool invocation	Once per invoked tool
Chat middleware	Request to the model client	Every model call; often several per run

All types support both function-based and class-based implementations

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/?tabs=python

Agent Middleware

- Agent middleware intercepts and modifies agent run execution
  - Uses AgentContext
- Agent-level middleware wraps run-level middleware
- Function/chat middleware follows the same wrapping principle

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/agent-vs-run-scope?tabs=python

Agent vs Run Scope

Agent-level

- Configured once on the Agent
- Applies to every run
- Best for baseline security, telemetry, and policy

Run-level

- Passed to one agent.run(...)
- Applies only to that invocation
- Best for targeted diagnostics or per-request customization

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/agent-vs-run-scope?tabs=python

Middleware executes like an onion

Request path

- A1 enters, then A2
- R1 enters, then R2
- Agent executes at the center

Response path

- R2 exits, then R1
- A2 exits, then A1
- The outermost layer finishes last

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/?tabs=python

D E M O

~5 min

The onion, printed

Run the official agent_and_run_level_middleware.py sample live: printed enter/exit lines match the previous slide's onion diagram exactly — agent-level middleware outermost, run-level middleware inside it, the agent execution at the center.

Runbook: demos/day3/module-4-demo-1-onion-order.md

Source: https://github.com/microsoft/agent-framework/blob/main/python/samples/02-agents/middleware/agent_and_run_level_middleware.py

Coordinate through context.metadata

Outer middleware

Set correlation ID and start time

Inner middleware

Read or enrich shared metadata

Tool/model interceptors

Attach trace context and decisions

Outer unwind

Read results and emit one summary

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/shared-state?tabs=python

Logging and timing example

```
async def timing(context, call_next):
    started = perf_counter()
    context.metadata["trace_id"] = new_trace_id()
    try:
        await call_next()
    finally:
        record_duration(
            perf_counter() - started,
            context.metadata["trace_id"],
        )
```

Then reference the middleware when creating your agent: middleware=[(add your middleware here)]

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/defining-middleware?tabs=python

Guardrails can short-circuit

1 Inspect

Evaluate request, identity, quota, or policy

2 Allow

Await call_next() when the request is permitted

3 Set result

Supply a controlled AgentResponse when blocked

4 Raise

MiddlewareTermination ends the pipeline deliberately

In Python, middleware stops execution by setting context.result when needed and raising MiddlewareTermination, or by short-circuiting the chain without calling call_next()

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/termination?tabs=python

Termination has an explicit result

```
if blocked(context.messages):
    context.result = AgentResponse(
        messages=[Message("assistant", ["I can't process this request."])]
    )
    raise MiddlewareTermination()

await call_next()
```

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/termination?tabs=python

D E M O

~4 min

Guardrail blocks a request, live

Run the official atr_validation_middleware.py sample live: a benign query passes through normally, then a query matching a deny-list rule is blocked before the tool executes — context.result set, MiddlewareTermination raised, exactly as the previous slide's code shows.

Runbook: demos/day3/module-4-demo-2-guardrail-termination.md

Source: https://github.com/microsoft/agent-framework/blob/main/python/samples/02-agents/middleware/atr_validation_middleware.py

Handle exceptions where you can add value

Failure	Layer	Response
Invalid tool arguments	Function	Reject or normalize before execution
Transient model/service error	Chat or agent	Bounded retry if the operation is safe
Policy violation	Agent or function	Controlled result + termination
Unknown exception	Nearest useful layer	Record, clean up, re-raise

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/exception-handling?tabs=python

Retry must be bounded and idempotent

Classify

Retry only known transient failures

Check safety

Read or idempotent operation?

Back off

Delay with a fixed maximum attempt count

Stop

Surface the last failure; never loop forever

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/exception-handling?tabs=python

Built-in OTel is not custom middleware

Built-in observability

- Standard OpenTelemetry integration
- Agent/model/tool spans and metrics
- Prefer for baseline telemetry

Custom middleware

- Domain policy and validation
- Result transformation or short-circuit
- Add only what standard instrumentation does not provide

Source: https://learn.microsoft.com/en-us/agent-framework/agents/observability?tabs=python · +1 in notes

Ordering and performance trade-offs

- **Outermost first** — Authentication and cheap rejection
- **Per-call cost** — Chat and function middleware can run many times
- **Mutation risk** — Document which layer changes messages, options, or results
- **Streaming** — Preserve streaming behavior; do not buffer unless required

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/?tabs=python

Experimental: Agent Hooks

Normal middleware

- General interception and composition
- Application specific ordering and behavior
- Core pattern for this workshop

Agent Hooks — EXPERIMENTAL

- Apply standard governance and runtime controls at well-defined points
- Implements Agent Hooks Specification
- Control plane, not telemetry plane
- Interceptors return a verdict that can optionally be enforced

Source: https://learn.microsoft.com/en-us/agent-framework/agents/agent-hooks?tabs=python

Takeaways

- You place controls at agent, function, or chat frequency intentionally.
- You use agent-level scope for baselines and run-level scope for one request.
- You reason about onion order before you register middleware.
- You set context.result and raise MiddlewareTermination to short-circuit.
- You keep retries bounded, classified, and safe for side effects.

Source: https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/?tabs=python